1.Deletion at beginning.

main.c Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void deleteAtStart(struct Node **head) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = *head;
    *head = (*head)->next;
    free(temp);
}

void display(struct Node *head) {
    struct Node *temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node *head, *first, *second, *third;
```

Output

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting first node:
20 -> 30 -> 40 -> NULL

=== Code Execution Successful ===
```

main.c Share Run

```c
    struct Node *head, *first, *second, *third;

    head = (struct Node*)malloc(sizeof(struct Node));
    first = (struct Node*)malloc(sizeof(struct Node));
    second = (struct Node*)malloc(sizeof(struct Node));
    third = (struct Node*)malloc(sizeof(struct Node));

    head->data = 10;
    head->next = first;

    first->data = 20;
    first->next = second;

    second->data = 30;
    second->next = third;

    third->data = 40;
    third->next = NULL;

    printf("Original List:\n");
    display(head);

    deleteAtStart(&head);

    printf("After deleting first node:\n");
    display(head);

    return 0;
}
```

Output

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting first node:
20 -> 30 -> 40 -> NULL

=== Code Execution Successful ===
```

2.Deletion at End.

main.c

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void deleteAtEnd(struct Node **head) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }

    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }

    struct Node *temp = *head;

    while (temp->next->next != NULL) {
        temp = temp->next;
```

Output

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting last node:
10 -> 20 -> 30 -> NULL


=== Code Execution Successful ===
```

main.c

```c

    free(temp->next);
    temp->next = NULL;
}

void display(struct Node *head) {
    struct Node *temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node *head, *first, *second, *third;

    head = (struct Node*)malloc(sizeof(struct Node));
    first = (struct Node*)malloc(sizeof(struct Node));
    second = (struct Node*)malloc(sizeof(struct Node));
    third = (struct Node*)malloc(sizeof(struct Node));

    head->data = 10;
    head->next = first;
```

Output

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting last node:
10 -> 20 -> 30 -> NULL


=== Code Execution Successful ===
```

main.c

```c
    head->data = 10;
    head->next = first;

    first->data = 20;
    first->next = second;

    second->data = 30;
    second->next = third;

    third->data = 40;
    third->next = NULL;

    printf("Original List:\n");
    display(head);

    deleteAtEnd(&head);

    printf("After deleting last node:\n");
    display(head);

    return 0;
}
```

Output

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting last node:
10 -> 20 -> 30 -> NULL

=== Code Execution Successful ===
```

3.Deletion at given position.

main.c

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void deleteAtPosition(struct Node **head, int position) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = *head;

    if (position == 1) {
        *head = temp->next;
        free(temp);
        return;
    }

    for (int i = 1; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
```

Output

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting node at position 3:
10 -> 20 -> 40 -> NULL

=== Code Execution Successful ===
```

main.c Share Run

```c
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Invalid position\n");
        return;
    }

    struct Node *nodeToDelete = temp->next;
    temp->next = nodeToDelete->next;
    free(nodeToDelete);
}

void display(struct Node *head) {
    struct Node *temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node *head, *first, *second, *third;

```

Output

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting node at position 3:
10 -> 20 -> 40 -> NULL

=== Code Execution Successful ===
```

main.c Share Run

```c
    head = (struct Node*)malloc(sizeof(struct Node));
    first = (struct Node*)malloc(sizeof(struct Node));
    second = (struct Node*)malloc(sizeof(struct Node));
    third = (struct Node*)malloc(sizeof(struct Node));

    head->data = 10;
    head->next = first;

    first->data = 20;
    first->next = second;

    second->data = 30;
    second->next = third;

    third->data = 40;
    third->next = NULL;

    printf("Original List:\n");
    display(head);

    int position = 3;
    deleteAtPosition(&head, position);

    printf("After deleting node at position %d:\n", position);
    display(head);

    return 0;
}
```

Output

```
Original List:
10 -> 20 -> 30 -> 40 -> NULL
After deleting node at position 3:
10 -> 20 -> 40 -> NULL

=== Code Execution Successful ===
```